

SYSTEM ANALYSIS AND DESIGN

Unit-IV: Detail Design, System Control & Quality Assurance

Comprehensive Study Notes

Himachal Pradesh University · BCA Semester IV

UNIT-IV: DETAIL DESIGN, SYSTEM CONTROL & QUALITY ASSURANCE

1. Detail Design

1.1 What is Detail Design?

Detail Design is the phase where the system design is converted into detailed specifications that can be used for implementation. It involves designing modules, files, databases, and user interfaces.

1.2 Objectives of Detail Design

Objective	Description
Specification	Provide detailed specifications for implementation
Modularity	Divide the system into modules
Reusability	Design components that can be reused
Maintainability	Make the system easy to maintain
Performance	Ensure performance requirements are met
Security	Address security requirements

2. Modularization

2.1 What is Modularization?

Modularization is the process of dividing a system into smaller, manageable, and independent modules that can be developed, tested, and maintained separately.

2.2 Characteristics of Good Modules

Characteristic	Description
Cohesion	Single, well-defined purpose
Coupling	Low coupling between modules
Independence	Can be developed independently
Reusability	Can be reused in other systems
Maintainability	Easy to maintain and modify
Testability	Easy to test individually

2.3 Advantages of Modularization

Advantage	Description
Ease of Development	Parallel development possible
Simplified Testing	Testing individual modules easier
Maintainability	Easier to maintain and modify
Reusability	Can be reused in different projects
Error Isolation	Errors can be isolated to specific modules

Team Development

Different teams can work on different modules

3. Module Specification

3.1 What is Module Specification?

Module Specification is a detailed description of a module's functionality, inputs, outputs, and behavior.

3.2 Module Specification Components

Component	Description
Module Name	Unique identifier for the module
Purpose	What the module does
Inputs	Data received by the module
Outputs	Data produced by the module
Processing Logic	How the module works
Error Handling	How errors are handled
Dependencies	Other modules needed

4. Cohesion & Coupling

4.1 What is Cohesion?

Cohesion refers to the degree to which the elements inside a module belong together. A highly cohesive module has a single, well-defined purpose and all its elements contribute to that purpose.

Types of Cohesion (from worst to best)

Type	Description	Rank
------	-------------	------

Coincidental	Elements grouped arbitrarily with no meaningful relationship	Worst
Logical	Elements related by logic (e.g., all input operations)	Poor
Temporal	Elements grouped because they occur at the same time	Low
Procedural	Elements related by sequence of execution	Medium
Communicational	Elements operating on the same data	Good
Sequential	Output of one element is input to another	High
Functional	All elements contribute to a single well-defined function	Best

4.2 What is Coupling?

Coupling refers to the degree of interdependence between modules. Low coupling (loose coupling) is desirable as it reduces the impact of changes in one module on other modules.

Types of Coupling (from worst to best)

Type	Description	Rank
Content Coupling	One module directly modifies data inside another module	Worst
Common Coupling	Modules share global data	Poor
External Coupling	Modules share external formats or interfaces	Low
Control Coupling	One module controls the logic of another via flags	Medium
Stamp Coupling	Modules share a composite data structure	Fair
Data Coupling	Modules share only simple data via parameters	Best

4.3 Cohesion vs Coupling Comparison

Aspect	Cohesion	Coupling
Definition	Intra-module binding (within a module)	Inter-module binding (between modules)
Desired Level	High cohesion — elements strongly related	Low coupling — modules loosely connected
Impact of Bad Design	Module does multiple unrelated tasks	Changes in one module affect many others
Relationship	High cohesion leads to low coupling	Low coupling is enabled by high cohesion

5. File Design

5.1 What is File Design?

File Design is the process of designing the structure and organization of files used in the system.

5.2 File Organization Types

Type	Description	Advantages	Disadvantages
Sequential	Records stored in sequential order	Simple, efficient for batch processing	Slow access for individual records
Indexed Sequential	Ordered records with an index for faster access	Fast access, supports sequential and direct access	Complex, overhead of maintaining index
Direct / Random	Records accessed directly using a key	Fast retrieval of individual records	Wasted space, complex hashing

6. Database Design

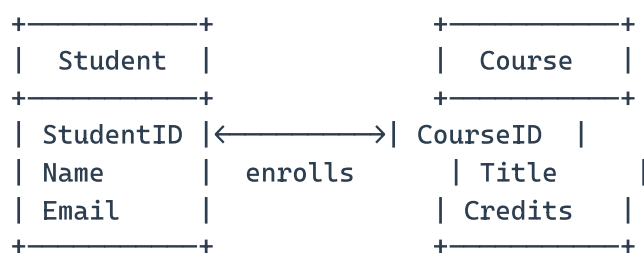
6.1 Database Design Process

Database design is the process of structuring data according to a database model. It involves defining the structure, storage, and retrieval mechanisms for data.

6.2 Database Design Steps

Step	Description
1. Requirement Analysis	Identify data requirements
2. Conceptual Design	Create Entity-Relationship (ER) Diagram
3. Logical Design	Convert ER diagram to relational schema
4. Physical Design	Implement in DBMS with tables, indexes, constraints
5. Implementation	Create tables, indexes, and constraints in the database

6.3 ER Diagram Example



6.4 Relational Schema Example

```

Student(StudentID, Name, Email, Phone, Address)
Course(CourseID, Title, Credits, Department)
Enrollment(StudentID, CourseID, Semester, Grade)
  
```

```
StudentID ref Student(StudentID)  
CourseID ref Course(CourseID)
```

7. Design Tools

7.1 Overview of Design Tools

+-----+	+-----+
STRUCTURE CHARTS	WARNIER-ORR DIAGRAM
+-----+	+-----+
Hierarchical module	Hierarchical data
decomposition	and process structure
+-----+	+-----+
+-----+	+-----+
PSEUDOCODE	FLOWCHARTS
+-----+	+-----+
Plain English logic	Visual program flow
description	with standard symbols
+-----+	+-----+

7.2 Structure Charts

A **Structure Chart** is a hierarchical diagram that shows the decomposition of a system into modules, their relationships, and the flow of data and control between them.

- Rectangles represent modules
- Arrows with empty circles represent data couples
- Arrows with filled circles represent control flags
- Shows the call hierarchy of modules

7.3 Warnier-Orr Diagram

The **Warnier-Orr Diagram** is a hierarchical diagramming technique that uses brackets to show the logical structure of data and processes. It represents systems using three constructs: sequence, selection, and iteration.

7.4 Flowcharts

Flowcharts are graphical representations of an algorithm or process using standard symbols connected by arrows.

Symbol	Name	Purpose
Oval	Terminator	Start or End of program
Parallelogram	Input/Output	Data input or output
Rectangle	Process	Processing operation
Diamond	Decision	Conditional branching
Arrow	Flow Line	Direction of flow

7.5 Pseudocode

Pseudocode is a plain language description of the steps in an algorithm or program logic. It uses structured programming constructs but is not tied to any specific programming language syntax.

```
PROCEDURE CalculateGrade(marks)
  IF marks ≥ 90 THEN
    grade = "A"
  ELSE IF marks ≥ 75 THEN
    grade = "B"
  ELSE IF marks ≥ 60 THEN
    grade = "C"
  ELSE IF marks ≥ 50 THEN
    grade = "D"
  ELSE
    grade = "F"
  ENDIF
  RETURN grade
END PROCEDURE
```

7.6 Documentation Tools

Tool	Purpose
Flowcharts	Visual representation of program flow

Pseudocode	Plain English description of logic
Structure Charts	Hierarchical representation of modules
Data Flow Diagrams	Data movement through the system
Entity-Relationship Diagrams	Database design

7.7 Documentation Types

Type	Description
System Documentation	Overall system description
Technical Documentation	Technical details for developers
User Documentation	Manuals for users
Operational Documentation	Procedures for operators

8. System Control

8.1 What is System Control?

System Control refers to the mechanisms and procedures used to ensure the system operates correctly and securely. It encompasses input controls, processing controls, and output controls.

8.2 Input Controls

Control Type	Description
Data Validation	Checking data for correctness (format, range, type)
Input Format Checks	Ensuring data follows specified format
Range Checks	Verifying values fall within acceptable ranges
Completeness Checks	Ensuring all required fields are entered
Uniqueness Checks	Verifying no duplicate entries exist

8.3 Processing Controls

Control Type	Description
Run-to-Run Controls	Reconciling input and output totals across runs
Reasonableness Checks	Verifying computed results are reasonable
Sequence Checks	Ensuring transactions are processed in order
Audit Trails	Logging processing activities for review

8.4 Output Controls

Control Type	Description
Output Verification	Comparing output against expected results
Access Controls	Restricting who can view or print outputs
Output Distribution	Controlled distribution of reports and files
Data Encryption	Protecting sensitive output data

8.5 Design Objectives: Reliability & Maintainability

Objective	Description
Reliability	System should function correctly and consistently
Maintainability	System should be easy to maintain and modify
Availability	System should be available when needed
Scalability	System should handle growth
Security	System should be secure from threats

Reliability Aspects

Aspect	Description
Fault Tolerance	System continues to operate despite failures
Error Detection	Ability to detect errors
Error Recovery	Ability to recover from errors
Redundancy	Backup components for critical parts

Maintainability Aspects

Aspect	Description
Modularity	Easy to modify individual modules
Documentation	Well-documented system
Readability	Code and design easy to understand
Testability	Easy to test changes

8.6 Control Objectives

Objective	Description
Data Integrity	Ensure data accuracy and consistency
Security	Protect against unauthorized access
Availability	Ensure system is available when needed
Confidentiality	Protect sensitive information
Auditability	Ability to track system activities

8.7 Control Mechanisms

Mechanism	Description
Access Control	Restrict access based on roles
Authentication	Verify user identity
Authorization	Control what users can do
Audit Trails	Record system activities
Backup and Recovery	Regular backups and recovery procedures

Encryption	Encrypt sensitive data
Firewalls	Protect network access

9. Quality Assurance

9.1 What is Quality Assurance?

Quality Assurance (QA) is a systematic process of ensuring that the system meets specified quality standards and requirements. It encompasses all activities from planning to implementation that contribute to building a high-quality system.

9.2 Quality Factors

Factor	Description
Correctness	System meets its specifications and user needs
Reliability	System performs consistently without failure
Efficiency	Optimal use of resources (CPU, memory, storage)
Integrity	System protects data from unauthorized access
Usability	Ease of learning and using the system
Maintainability	Ease of making changes and fixing issues
Testability	Ease of testing the system
Flexibility	Ability to adapt to changing requirements
Portability	Ability to run on different platforms/environments
Reusability	Components can be used in other systems

10. Testing

10.1 Unit Testing

Unit Testing tests individual components or modules in isolation. It verifies that each module functions correctly according to its specification.

10.2 Integration Testing

Integration Testing tests the interaction between multiple modules. It ensures that modules work together correctly when combined.

10.3 System Testing

System Testing tests the complete integrated system to verify that it meets all specified requirements.

10.4 Acceptance Testing

Acceptance Testing is performed by end-users to determine whether the system meets their needs and should be accepted for deployment.

10.5 Testing Strategies

Strategy	Description
Top-Down	Start with main modules and add sub-modules progressively
Bottom-Up	Start with sub-modules and combine upward
Big Bang	Combine all modules at once and test
Incremental	Add modules one by one, testing at each step

10.6 Types of Testing

Type	Description
Unit Testing	Testing individual components
Integration Testing	Testing interactions between components
System Testing	Testing the complete system
Acceptance Testing	Testing by users
Regression Testing	Testing after changes to ensure existing functionality works
Performance Testing	Testing speed and response time
Security Testing	Testing security measures
User Acceptance Testing (UAT)	Final testing by end-users

10.7 White Box vs Black Box Testing

Aspect	White Box Testing	Black Box Testing
Also Known As	Glass box, structural, or code-based testing	Behavioral or specification-based testing
Focus	Internal structure and code logic	External behavior and functionality
Performed By	Developers	Testers or end-users
Techniques	Statement coverage, branch coverage, path testing	Equivalence partitioning, boundary value analysis
Knowledge Required	Knowledge of internal code	No knowledge of internal code needed

Goal	Verify internal logic and coverage	Verify functionality against requirements
-------------	------------------------------------	---

10.8 Test Plan Components

Component	Description
Scope	What will be tested
Strategy	Approach to testing
Schedule	When testing will occur
Resources	People and tools needed
Test Cases	Specific test scenarios
Success Criteria	When testing is complete
Reporting	How results will be reported

11. System Implementation

11.1 Implementation Strategies

Strategy	Description	Advantages	Disadvantages
Direct Cutover	Old system is turned off and new system is immediately turned on	Simple, no parallel costs	High risk; no fallback if new system fails
Parallel	Old and new systems operate simultaneously	Low risk; can compare outputs	Expensive, double workload
Pilot	New system is implemented in one part of the organization first	Low risk, manageable scope	May not cover all scenarios
Phased	System is implemented in stages/modules	Controlled rollout, manageable	Takes longer to implement fully

12. System Maintenance

12.1 Types of System Maintenance

Type	Description	Example
Corrective Maintenance	Fixing errors and bugs discovered after implementation	Fixing a calculation error in payroll system
Adaptive Maintenance	Modifying the system to adapt to changes in the environment	Updating system for new operating system version
Perfective Maintenance	Enhancing the system to improve performance or add features	Adding new report formats, improving UI
Preventive Maintenance	Proactive changes to prevent future issues	Code refactoring, database optimization

13. Questions for Practice

Short Answer Questions (2-5 Marks)

1. What is detail design?
2. What is modularization?
3. What is module specification?
4. What is file design?
5. What are the types of file organization?
6. What is unit testing?
7. What is integration testing?
8. What is system testing?
9. What is acceptance testing?
10. What are the design objectives?

Long Answer Questions (10 Marks)

1. Explain the detail design process with its components.
2. Explain modularization and its advantages.
3. Explain file design and types of file organization.
4. Explain database design with ER diagram example.
5. Explain unit testing and integration testing.
6. Explain system control and quality assurance.

Objective Questions (1 Mark)

1. Modularization divides system into:
(a) Modules (b) Files (c) Databases (d) None
2. Unit testing tests:
(a) Individual modules (b) Multiple modules (c) Complete system (d) None

3. Integration testing tests:

(a) Individual modules (b) Module interactions (c) Complete system (d) None

4. File organization types include:

(a) Sequential (b) Indexed (c) Direct (d) All

5. ER diagram is used for:

(a) Database design (b) File design (c) Module design (d) None

End of Unit-IV Notes · SYSTEM ANALYSIS AND DESIGN · BCA Sem IV (HPU)